总是或者给出正确的解
或者无解

拉斯维加斯算法 Las vegas 算法

随机的次运行算法，每次相对独立
成功非正确 → 概率小.

Monte Carlo 算法

总给出答案.
偶尔给出不正确解.

算法设计与分析开溜.

① 算法的基本属性

0个或多个　至少一个

性质：输入、输出、可行性、确定性、有穷性.

优劣：时间/空间复杂度

特：[illegible] 有限多 [illegible] → 有限时间内完成

算法：对问题

一、填空题（每空格 1 分，共 20 分）

1. 一个算法的优劣可以用它的 时间复杂度 和 空间复杂度 来衡量。

2. 堆这种数据结构在逻辑上是 树 ，在物理上是 数组 。

3. 递归的两个必要条件，一是 边界条件 ，二是 递归模式 。

4. 对比分治策略与动态规划策略：两者分析问题的解的形式都是 递归 的，分治策略的算法的求解过程是 自顶向下 的，而动态规划策略的算法求解过程是 自底向上 ；分治策略的算法的子问题都是 相互独立的 ，而动态规划策略的子问题是 相互依赖的 。

5. 所谓的最优子结构性质的含义是 问题中的最优解包含着其每一个子问题的最优解 。

6. 构造最小生成树的 Prim 算法是一种贪心策略算法，贪心的性质表现在 选择权重最小的边加入当前最小生成树 。该算法 能 （填“能”或“不能”）求得问题的最优解。

7. 对比回溯法与分支限界法：回溯法按照 深度优先 策略搜索问题的解空间树；分支限界法按照 广度优先 策略搜索问题的解空间树。回溯法求解得到的是问题的可行解，而分支限界法求解得到的往往是问题的 最优解 ，而且需要预先定义一个 界限函数 限界函数 。

8. 拉斯维加斯型随机算法可能会失败，但是求解得到的解 是正确的 ；蒙特卡洛型随机算法总是能给出一个解，但却无法知道这个解 正确与否 。

二、单项选择题(每题 2 分，共 20 分)

1. 设某算法的时间复杂度为 $f(n)= an^2+n\log n+c$，这里 $a>0$，则以下选项哪个描述 f(n)的阶最恰当？ C

   A. $f(n)=O(n^2)$　　B. $f(n)=\Omega(n^2)$　　C. $f(n)=\Theta(n^2)$　　D. $f(n)=o(n^2)$

2. 算法是由若干条指令组成的有序序列，下面哪一项不是算法应该具有的基本属性？ D

   A. 具有零个或多个外部量作为输入 输入　　B. 算法的每条指令都是清晰、无歧义的 确定性

   C. 至少具有一个量作为输出 输出　　D. 算法复杂度在输入规模足够大时具有收敛性 有穷

3. 秦始皇吞并六国一统天下，使用了远交近攻、逐个击破的连横策略，采用的哪种算法思想？ C

   A. 迭代　B.递归　C.分治　D.回溯　　分开击破

4. 下列哪个选项不属于分析算法复杂度的方法？ A

A. 编程实现并测量其运行时间　　B. 计算迭代次数

C. 分析基本运算频度　　D. 使用递推关系

5. 哈夫曼编码可以利用__C__算法来实现。

A. 分治策略　B.动态规划策略　C.贪心策略　D.回溯法

6. 设问题规模为n，下列排序算法中，哪个算法的(空间复杂度)最大？ B

A. 归并排序 $O(n\log n)$　B. 冒泡排序 $O(n^2)$　C. 堆排序 $O(n\log n)$　D. 快速排序 $O(n\log n)$

7. 关于贪心策略，以下说法成立的是__A__

A. 贪心策略并非总是能找到问题的最优解　　B. 贪心策略的计算效率往往很低

C. 贪心策略总是能找到理论上最优解　　D. 贪心策略等价于蛮力法求解

8. 考虑图 G 的 3 着色问题：设 G 共有 n 个顶点，用 3 种颜色之一为 G 每个顶点着色，使得任意相邻顶点具有不同的颜色。若用蛮力方法求解，解空间搜索树的叶子结点个数有__B__个。

A. 3n　B. 3^n　C. n!　D. C_n^3

9. 下列哪种算法遵循的不是分治策略？ A

A. 二分查找算法　B. 快速排序算法　C. 堆排序算法　D. Strassen 矩阵乘法

10. 下列哪一个求解步骤不是分治策略求解问题时所必须的？ D

A. 将原问题分别分别为若干子问题

B.分别独立求解子问题

C.将子问题的最优解整合为原问题的最优解

D.根据计算最优值时得到的信息，构造最优解 动态规划

三、解答题(共 60 分)

1. （本题共 27 分）给定数组 A[1…n]， 设 $n=2^k$，k 是非负整数，需要求该数组的最大元素和最小元素，请按要求回答如下问题：

1) 输入：数组A[1…n]
输出：最大元素x，最小元素y

```
x ← A[1]
y ← A[1]
for i ← 2 to n
    x ← max(x, A[i])
    y ← min(y, A[i])
end for
return (x, y)
```

$T(n)=\sum_{i=2}^{n} 2 = 2(n-1) = \Theta(n)$

3) 输入：数组A[1,…n]
输出：最大元素x和最小元素y

1. maxmin (A, 1, n)

过程：maxmin(A, low, high)

```
1. if low - high = 1 then
    return (max(A[low], A[high]), min(A[low], A[high]))
else
    mid = ⌊(low+high)/2⌋
    (x1, y1) ← maxmin(A, low, mid)
    (x2, y2) ← maxmin(A, mid+1, high)
    x ← max(x1, x2)
    y ← min(y1, y2)
    return (x, y)
end if
```

$$T(n)=\begin{cases} 1 & n=2 \\ 2T(\frac{n}{2})+2 & n>2 \end{cases}$$

1) 给出一个比较朴素的算法，用伪代码/C/C++/Java 描述一下你的算法步骤。以元素的比较作为算法的元操作，该算法的时间复杂度是多少？（7 分）

2) 用分治策略解决这个问题，将 n 个元素按数组下标分成相等的两个子问题求解，以元素的比较作为算法的元操作，算法的时间复杂度可以描述为如下表达形式，请填写下式中的空白部分，并分析该算法的时间复杂度。（10 分）

$$T(n)=\begin{cases} 1 & if\ n=2 \\ 2T(n/2)+2 & if\ n>2 \end{cases}$$

令 $n=2^k$ $T(n)=2T(\frac{n}{2})+2$

$=2(2T(\frac{n}{2^2})+2)+2$

$=2^{k-1}T(n/2^{k-1})+\frac{2(2^{k-1}-1)}{2-1}$

$=2^{k-1}+2^k-2=\frac{3}{2}n-2=\Theta(n)$

3) 用伪代码/C/C++/Java 描述一下你的分治策略算法步骤。（10 分）

2. （本题 18 分）考虑如下最长公共子序列问题：

1）给定两个字符串 A 和 B，长度分别为 n 和 m。请写出利用动态规划的算法求解其最长公共子序列长度的递推规则。（8 分）

2）如果上述字符串 A=“xyxxzx”，B=“zxzyyz”，请用图表的形式写出利用动态规划算法求解其最长公共子序列长度的过程。（10 分）

1)

$$C[i,j]=\begin{cases} 0 & if\ i=0\ or\ j=0 \\ C[i-1,j-1]+1\ \text{“}\nwarrow\text{”} & if\ i>0\ and\ j>0\ and\ A[i]=B[j] \\ \max(C[i-1,j]\ \text{“}\uparrow\text{”},C[i,j-1]\ \text{“}\leftarrow\text{”}) & if\ i>0\ and\ j>0\ and\ A[i]\neq B[j] \end{cases}$$

2)

			x	y	x	x	z	x
		0	1	2	3	4	5	6
	0	0	0	0	0	0	0	0
z	1	0	0	0	0	0	1	1
x	2	0	1	1	1	1	1	2
z	3	0	1	1	1	1	2	2
y	4	0	1	2	2	2	2	2
y	5	0	1	2	2	2	2	2
z	6	0	1	2	2	2	3	3

最长公共子序列长度为 3

3. （本题 15 分）阅读如下求无向图 G=(V,E)的最小生成树的 Kruskal 算法伪代码，按要求作答：

1) 将该算法中空白部分用**伪代码**补充完整。（4 分）

2) 该算法的第 7、9 句的执行频度分别是多少？（6 分）

3) 给出如下无向图实例，如下图所示，用 Kruskal 算法给出该图的最小生成树。（5 分）

输入：无向图 G=(V,E)，设|V|=n，|E|=m

输出：该无向图的最小生成树 T

2) 第7句 至少 n-1次，至多 m次

第9句 n-1次 考完解

```
1.  对 E 中的边按权重以非降序排列
2.  for 每个顶点 v∈V
3.      MAKESET({v}) //构造集合，集合名为 v
4.  end for
5.  T= ____(1)____ {}
6.  while ____(2)____ |T|<n-1
7.      令(x,y)为 E 中的下一条边
8.      if  FIND(x) ≠ FIND(y)  then
9.          将边(x,y)加入 T
10.         UNION(x,y)  //将 x 和 y 合并为一个集合
```